University of Piraeus
Faculty of Information and Telecommunications Technologies
Department of Digital Systems

Level: Postgraduate Program of Study

Security of Wireless and Mobile Networks

Android-Use Case 1

Supervising Professors: Christos Xenakis and Georgios Efthymoglou
(Grigorios Papoutsis)

Kalaitzidis Ioannis ioannis_kalaitzidis@ssl-unipi.gr

Piraeus
04/05/2026

Abstract

Implementation of security control (penetration testing) and reverse engineering in two vulnerable Android applications with the aim of detecting hidden flags. The research included analysis through the Apktool tool to inspect critical settings files and through Android Studio to explore the internal file system. Through this process, the critical vulnerability of insecure local storage was highlighted, as the codes were detected without any encryption (plain-text). Specifically, the flags were successfully retrieved from the temporary SQLite (Write-Ahead Log) memory files of the first application and from the XML files of the Shared Preferences of the second.

Contents

Introduction

This study aims to evaluate security (Penetration Testing) in applications of the Android operating system. To achieve this, the technique of reverse engineering is used. Reverse engineering is like taking a finished product—that is, the finished application (file .apk)—and disassembling it step by step. The purpose of this process is to see what is hidden inside the application, how its code is written, and how exactly it works, in order to identify any weaknesses and security gaps. Research and analysis focus on two specific applications (Enumeration02.apk and Enumeration03.apk) and are divided into two main stages:

- **1st stage:** With the help of the **Apktool** tool, we "open" the files of the application. This allows us to inspect its configuration files and resources, looking for vulnerabilities, without having to run it.
- **2nd stage:** We install the applications on a virtual mobile phone through the **Android Studio** environment. While the application is running, we use the ADB (Android Debug Bridge) tool to penetrate the device's file system and check if the application stores sensitive data in an insecure way, such as in local databases or configuration files (Shared Preferences).

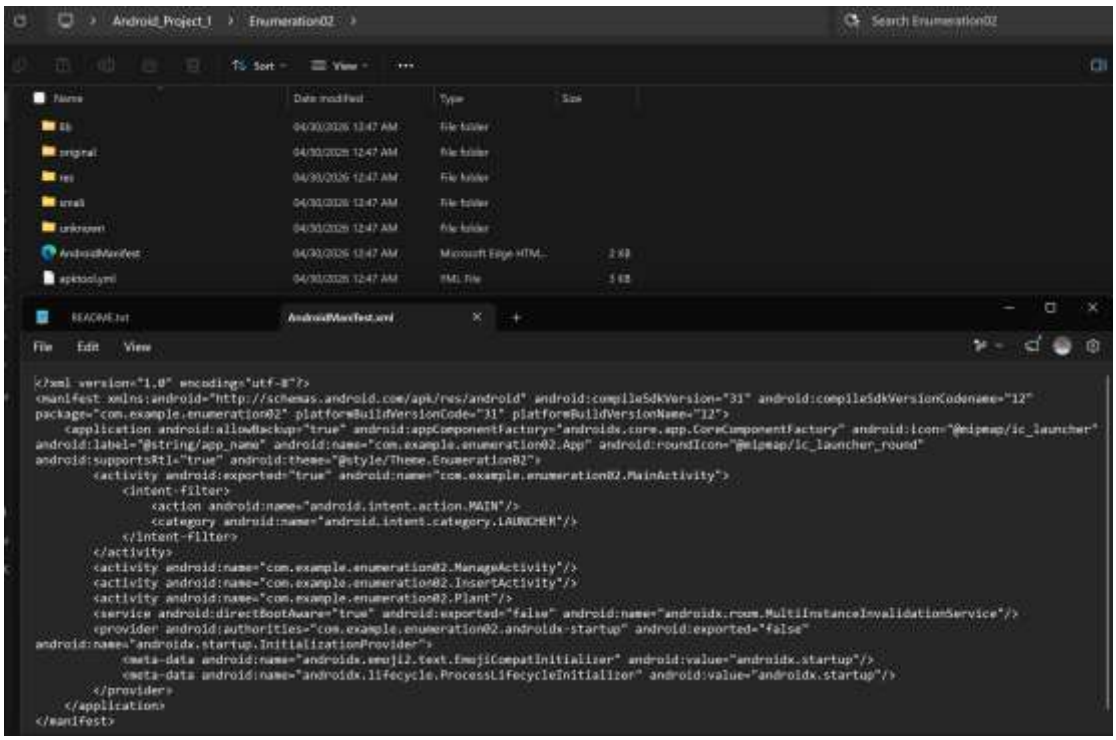
Often, developers rely solely on the protection provided by the operating system itself (sandboxing) and fail to encrypt the data they store locally (such as passwords). This is a critical security hole, which an attacker with physical access or root privileges to the device can easily exploit.

Practical piece of work

The first stage of the investigation involved the analysis of Enumeration02.apk application. As Android's installation files (APKs) are compressed, their contents could not be read directly. To circumvent this obstacle, the open-source tool was used **Apktool**. Through the command prompt, the decoding command "**apktool d**

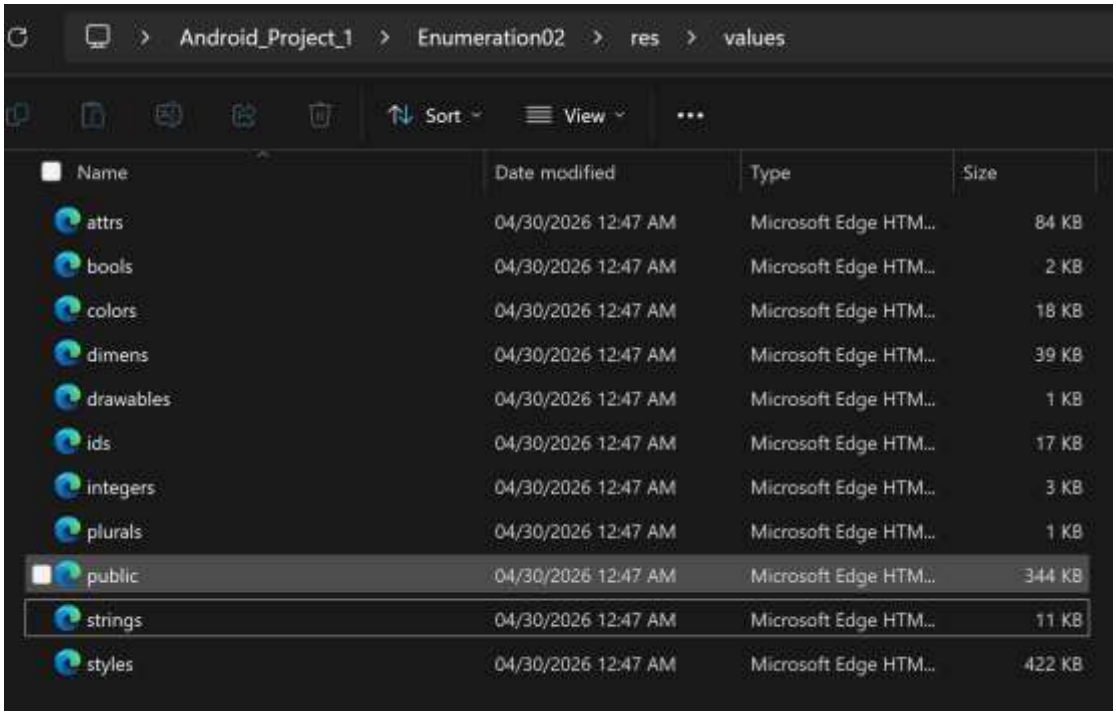
Enumeration02.apk", which undertook the dismantling of the application. This process "unlocked" the application, creating a new folder with all the code and settings files translated back to their original form. From this new folder, our analysis focused on Android's most critical settings file, the **AndroidManifest.xml**, which was opened in a simple word processor for further study of the permissions and hidden elements of the application.

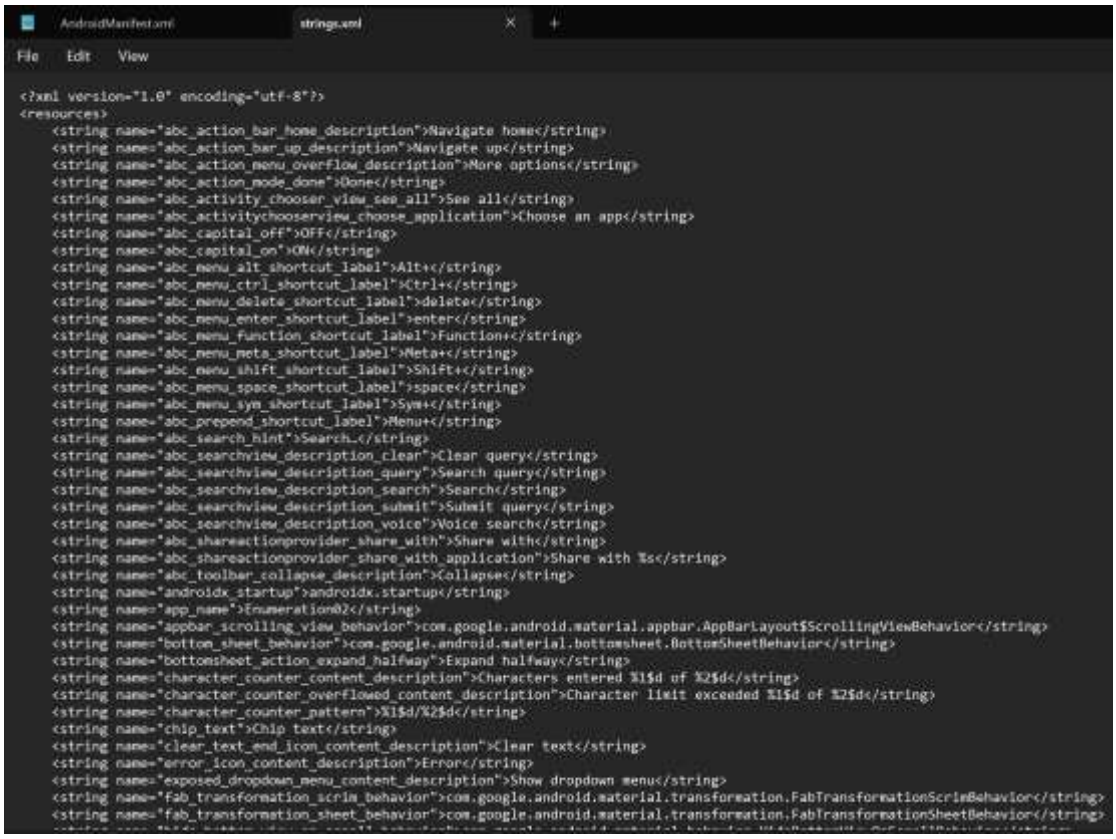
```
C:\Users\Kalai>cd Desktop

C:\Users\Kalai\Desktop>cd Android_Project_1

C:\Users\Kalai\Desktop\Android_Project_1>apktool d Enumeration02.apk
I: Using Apktool 3.0.1 on Enumeration02.apk with 8 threads
I: Loading resource table...
I: Baksmaling classes.dex...
I: Decoding value resources...
I: Loading resource table from file: C:\Users\Kalai\AppData\Local\apktool\framework\1.apk
I: Decoding file resources...
I: Generating values XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Copying original files...
I: Copying lib...
I: Copying unknown files...

C:\Users\Kalai\Desktop\Android_Project_1>
```



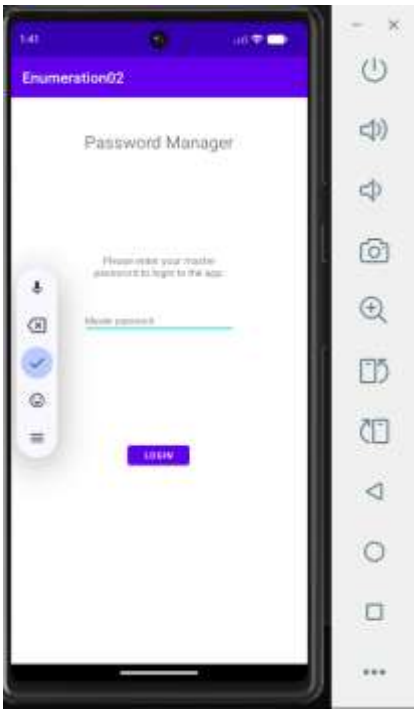
In reverse engineering, the analysis follows a hierarchy, starting with the configuration files where developers tend to leave data exposed. Initially, the **AndroidManifest.xml**. Although the file did not contain any flags in its metadata, its structure provided us with crucial information. The existence of the InsertActivity and ManageActivity screens has been revealed. This nomenclature strongly indicates the use of a local database in the background of the application.

The control was then transferred to the resources directory and specifically to the archive **res/values/strings.xml**. This file acts as the "dictionary" of the application and is a classic point where developers forget codes or API keys. In this case, too, the check was fruitless, as only standard UI texts (UI strings) were detected.



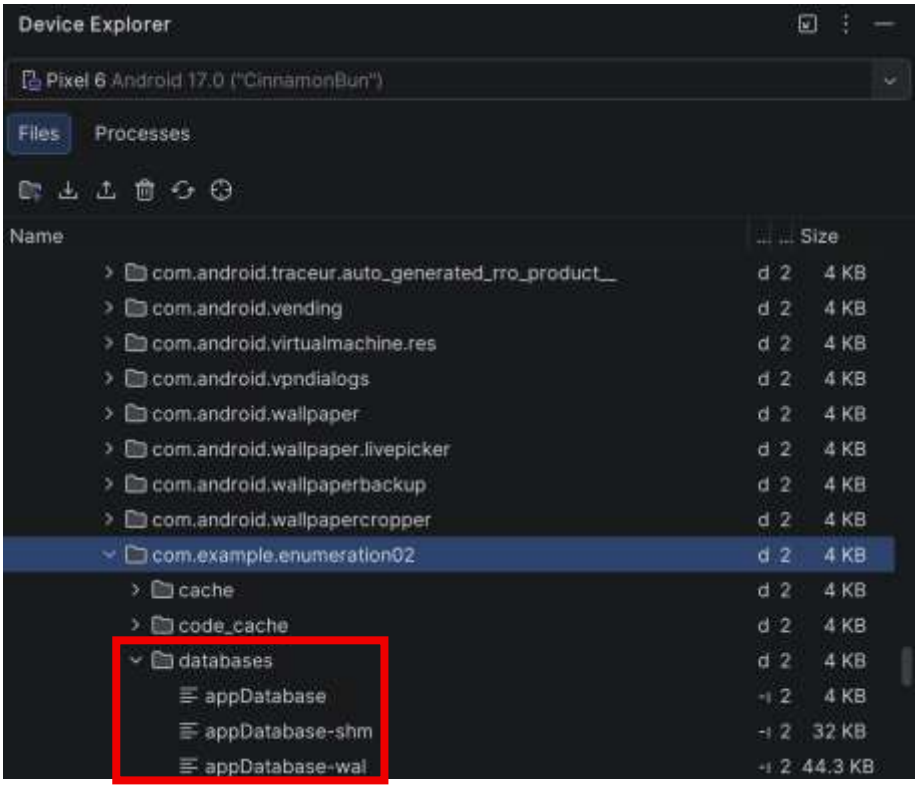


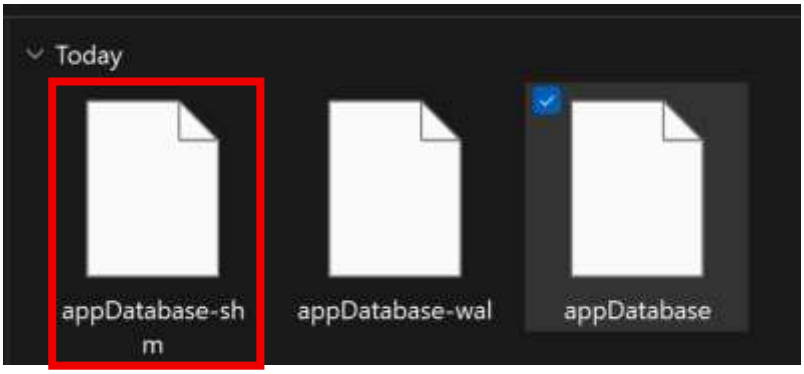
Since the basic configuration files did not reveal the flag in plain text format and the findings from the Manifest (existence of data management activities) lead us to the next step. The application will be installed and run in a virtual environment (Android Emulator) in order to live control the way it manages and stores its data (2The stage).



Using a virtual device provides us with the necessary root permissions to access the hidden system files. After the APK file was installed and executed, it was revealed to function as a Password Manager, requiring a 'Master password' for entry. This function confirms the hypothesis of the initial analysis (due to InsertActivity and ManageActivity), making it certain that the application uses some local database to store sensitive user codes.

To control how the codes are stored, we used Android Studio's Device Explorer tool. We navigated to the system directory (/data/data/), where each application stores its local data (sandboxing). Here it is worth noting that the database folder is not automatically created when installing the APK, but only after we run the application and interact with it. Having already run the application in the virtual environment, we successfully located its folder (com.example.enumeration02) and found the existence of the databases subfolder. As shown in the image below, a local SQLite database is stored there, which consists of the main file **appDatabase** and temporary logs (**appDatabase-shm** and **appDatabase-wal**). In particular, the -wal extension in the appDatabase-wal file stands for "Write-Ahead Log". On Android (and in SQLite databases in general), when an application saves new data, it first writes it to this temporary file, before permanently passing it to the main database file (appDatabase). In order to analyze their contents without the limitations of the device, these files were exported locally to the computer.

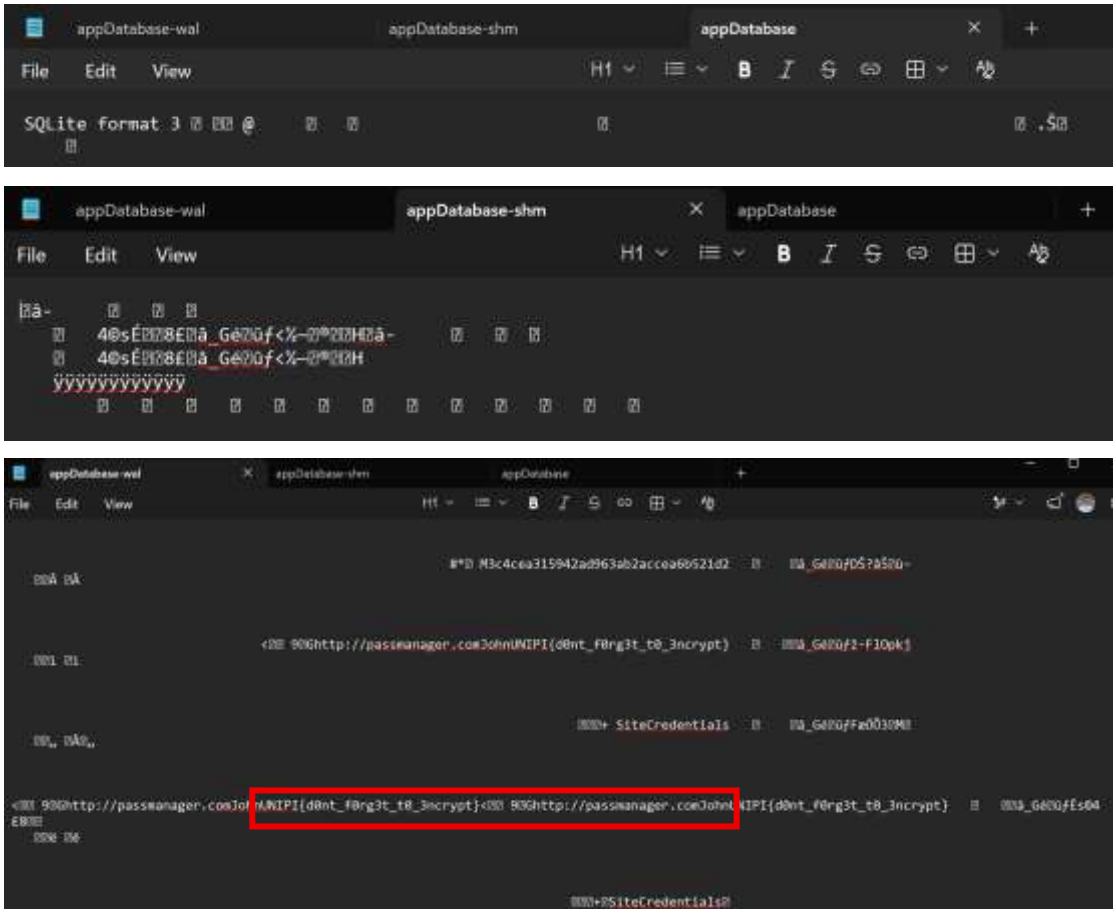




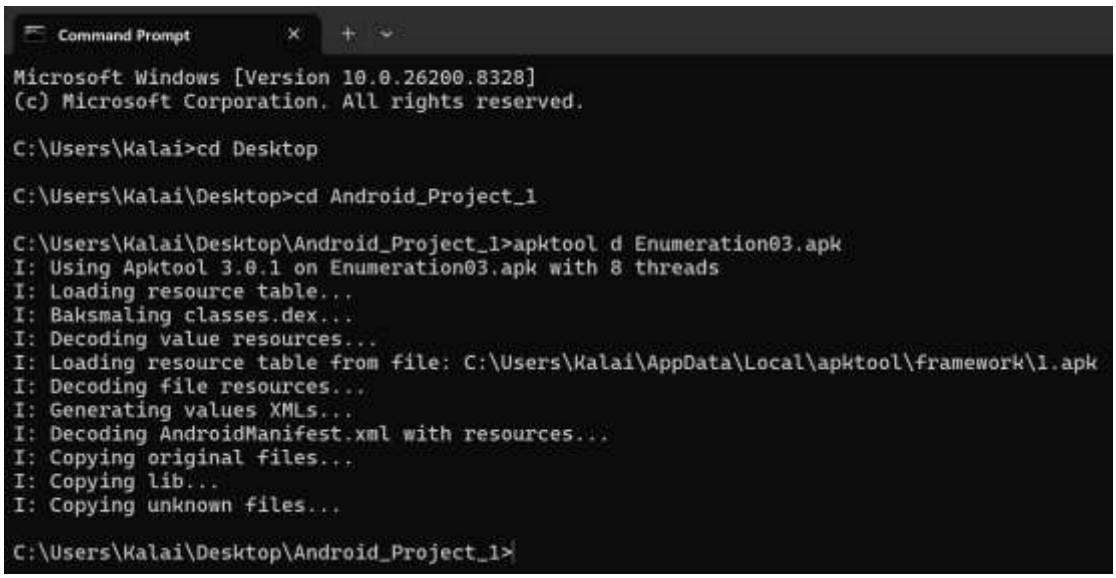
After exporting the files, the next step was to examine their contents. In order to check whether the application applies any encryption method to the sensitive data, the files were opened as raw text using a word processor (Notepad).

Upon inspection of the temporary appDatabase-wal (Write-Ahead Log), it was revealed that the app stores users' credentials completely unprotected. As can be seen in the image, among the binary data, the registration for the website (http://passmanager.com) with the username John was found in plain-text format. In the password field, the requested flag was clearly revealed: **UNIPI{d0nt_f0rg3t_t0_3ncrypt}**.

The absence of encryption in the local database allows any attacker with physical access - or root access - to the device, to intercept all the user's stored passwords with enormous ease. Therefore, with the flag, the penetration test for the specific application is successfully completed.

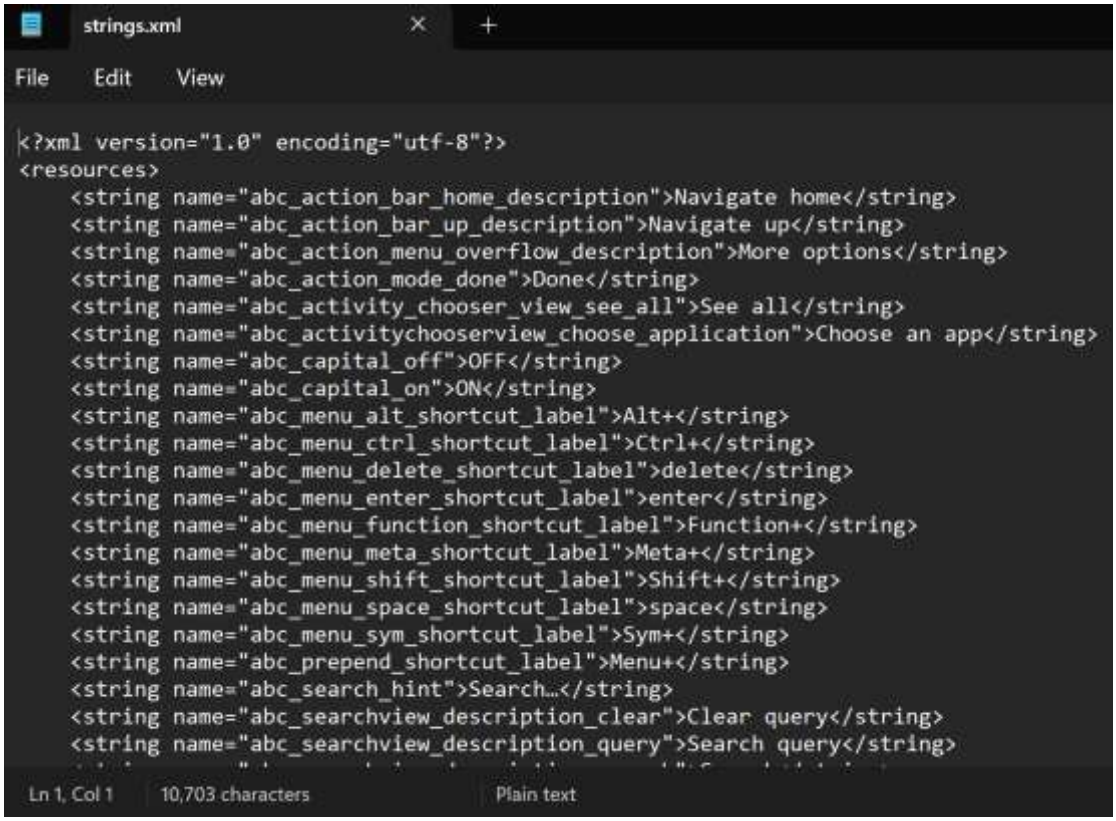


The evaluation of the second application (Enumeration03.apk) started following the same methodology. Using the Apktool tool, the installation file was decoded in order to check the key configuration files for any exposed data.



Initially, the AndroidManifest.xml file was examined. Although no flag was detected, the structure of the application was confirmed, as the activities

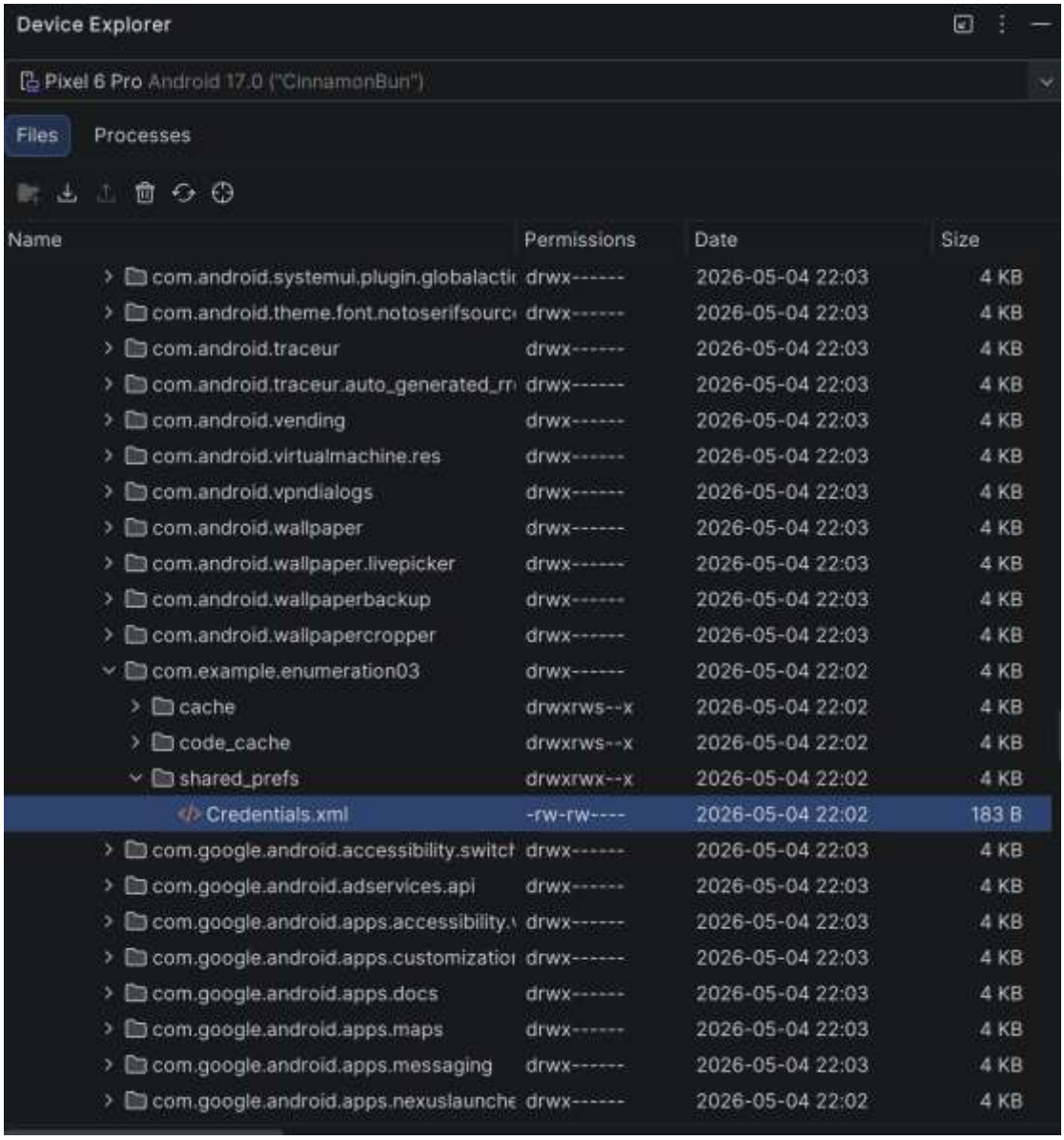
ManageActivity and InsertActivity were detected again. This strongly indicates the use of a local database in the background. The res/values/strings.xml resource file was then checked, which also turned out to be "clean".



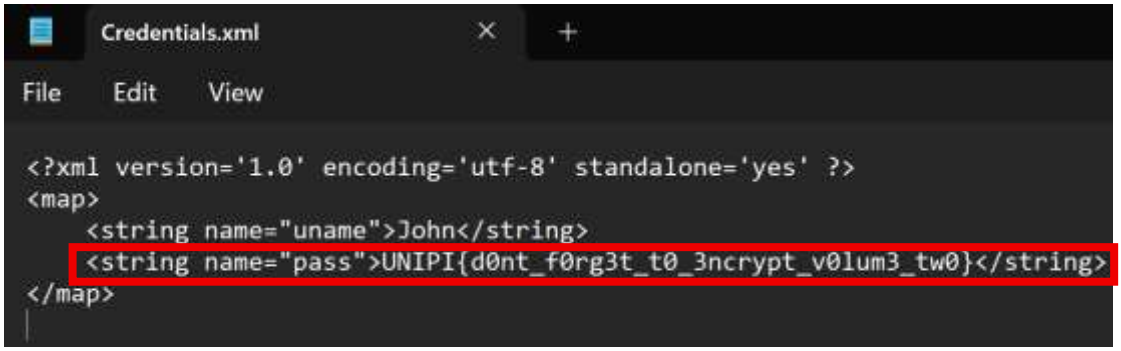
Since the analysis did not reveal any data in plain text, the next logical step is to switch to running the app in Android Studio in order to check how the app manages its data in real time.

The app Enumeration03.apk installed and running. The user interface (UI) confirmed our initial assumption, that the application functions as a Password Manager, as in the previous case. The similarity with the previous application is obvious, which leads us to check its file system to see if it repeats the same Insecure Data Storage errors or if it uses another, perhaps slightly more "cunning" method.

Following the process of dynamic analysis, we used the Device Explorer tool again to investigate the app's file system. We have navigated the list **/data/data/com.example.enumeration03**. This time, we found the databases folder missing. Instead, our attention turned to the file **shared_prefs** (Shared Preferences), which is another common storage point for settings on Android. The file was located inside the folder **Credentials.xml**.



The file was exported to the local computer and opened with a simple word processor (Notepad). As shown in the image, the file contains the records for the username (uname="John") and password. Once again, the data is stored in plain-text format. In place of the code, the requested flag was found: **UNIPI{d0nt_f0rg3t_t0_3ncrypt_v0lum3_tw0}**.



The application presents the exact same vulnerability as the previous one, but implemented with a different mechanism. The developer used Shared Preferences instead of SQLite database. This mechanism is designed exclusively for storing simple interface settings (e.g., display theme, language) and never for sensitive data. The XML files generated by Shared Preferences do not offer any native encryption, making intercepting the codes extremely easy in cases of device tampering.

Conclusion

This paper highlighted in a practical way the importance of penetration testing and reverse engineering in Android applications. Through the use of Apktool to disassemble and inspect code and Android Emulator to check the file system, a common mistake was identified, the misconception of protection. Very often, developers rely solely on the sandboxing mechanism of the Android operating system. However, as demonstrated in both applications under review (Enumeration02 and Enumeration03), any attacker gaining physical access or root privileges to the device can immediately bypass these restrictions and read the local files. The absence of encryption on sensitive data, such as passwords, leaves them exposed in plain-text format—whether they're hidden in temporary database memory files or simple configuration files.

Bibliography

- HackerSploit. (2020, February 22). *Penetration Testing Bootcamp*. Retrieved from https://www.youtube.com/watch?v=qJ9ZmkMvkcA

- JohnHammond. GitHub - JohnHammond/security-resources: A communal outpouring of online resources for learning different things in cybersecurity. Retrieved from https://github.com/JohnHammond/security-resources
- David Bombal. (2021, May 5). *CTF Walkthrough with John Hammond* [Video file]. Retrieved from https://www.youtube.com/watch?v=ZUqGSbvZp1k
- OWASP MASTG - OWASP Mobile Application Security. Retrieved from https://mas.owasp.org/MASTG/
- OWASP Mobile Top 10 | OWASP Foundation. Retrieved from https://owasp.org/www-project-mobile-top-10/
- Data and file storage overview. Retrieved from https://developer.android.com/training/data-storage
- Save simple data with SharedPreferences. Retrieved from https://developer.android.com/training/data-storage/shared-preferences
- Write-Ahead logging. Retrieved from https://sqlite.org/wal.html